МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: «Редакционное расстояние»
Вариант 1

Студент гр. 4343 _____ Толмачев И.А.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2026

Особо заменяемый и добавляемый символы

Задание.

"Особо заменяемый символ и особо добавляемый символ": цена замены определённого символа отличается от обычной цены замены; цена добавления другого (или того же) определённого символа отличается от обычной цены добавления. Особо заменяемый символ и цена его замены, особо добавляемый символ и цена его добавления — дополнительные входные данные.

Описание алгоритма и оценка его сложности.

Алгоритм основан на методе динамического программирования. Пусть даны две строки A длины n и B длины m, а также стоимости операций: базовая цена замены *base_replace*, базовая цена вставки *base_insert* и базовая цена удаления *base_delete*. Помимо этого, заданы особый символ замены *special_replace* (пара символ — цена) и особый символ вставки *special_insert* (пара символ — цена). Если операция применяется к особому символу, используется его персональная цена, иначе берётся базовая цена.

Строится двумерная таблица *dp* размером (n+1) x (m+1), где $dp[i][j]$ содержит минимальную стоимость преобразования первых i символов строки A в первые j символов строки B. Инициализация границ выполняется следующим образом:

$dp[i][0] = i * base_delete$ для всех i от 0 до n, превращение префикса A в пустую строку требует i удалений, каждое по базовой цене.

$dp[0][j]$ = dp[0][j-1] + *ins_cost*, где *ins_cost* равно *special_insert.second*, если $B[j-1]$ совпадает с *special_insert.first*, иначе равно *base_insert*. Таким образом, вставка символов из пустой строки учитывает особую цену для символа $B[j-1]$.

Заполнение таблицы для $i \geq 1$, $j \geq 1$ выполняется по правилам:

Если $A[i-1] == B[j-1]$, то $dp[i][j]$ = dp[i-1][j-1] (символы совпадают, операция не требуется).

Иначе вычисляются три варианта:

Замена $A[i\text{-}1]$ на B[j-1]: стоимость r*ep_cost* равна *special_replace.second*, если $A[i\text{-}1]$ равен *special_replace.first*, иначе *base_replace*. Тогда *replace* = $dp[i\text{-}1][j\text{-}1]$ + *rep_cost*.

Вставка символа $B[j\text{-}1]$: стоимость ins_cost равна special_insert.second, если $B[j\text{-}1]$ равен *special_insert.first*, иначе *base_insert*. Тогда *insert* = $dp[i][j\text{-}1]$ + *ins_cost*.

Удаление символа $A[i\text{-}1]$: стоимость всегда равна *base_delete,* так как удаления не имеют особых цен. Тогда *del* = $dp[i\text{-}1][j]$ + *base_delete*.

$dp[i][j] = min(\{replace, insert, del\})$.

По окончании заполнения таблицы минимальная стоимость преобразования *A* в *B* находится в $dp[n][m]$. Для восстановления последовательности операций выполняется обратный проход от $dp[n][m]$ к $dp[0][0]$. На каждом шаге проверяется равенство текущего значения $dp[i][j]$ одному из трёх выражений с учётом тех же цен, что использовались при заполнении:

Если символы равны, записывается 'M' и уменьшаются оба индекса.

Иначе, если $dp[i][j] == dp[i\text{-}1][j\text{-}1]$ + *rep_cost*, записывается 'R'.

Иначе, если $dp[i][j] == dp[i][j\text{-}1]$ + *ins_cost*, записывается 'I'.

Иначе, если $dp[i][j] == dp[i\text{-}1][j]$ + *base_delete*, записывается 'D'.

Если ни одно условие не выполнено, срабатывает аварийный выбор fallback, обеспечивающий завершение обхода.

Полученная в обратном порядке цепочка символов операций разворачивается, давая итоговую последовательность, применяемую слева направо.

Правило треугольника для весов операций гарантирует, что замена не хуже пары «удаление плюс вставка», и аналогично для комбинаций. Благодаря этому динамическое программирование всегда находит оптимальное решение, и восстановление пути не даёт противоречий.

Оценка сложности по времени. Основной этап — два вложенных цикла, заполняющих таблицу *dp* размером $(n+1)$ x $(m+1)$. Каждая итерация

выполняет не более трёх простых вычислений и обращений к словарям. Следовательно, временная сложность составляет O($n \cdot m$). Восстановление пути идёт за O(n+m), что не превосходит O($n \cdot m$). Общая временная сложность алгоритма — O($n \cdot m$).

Оценка сложности по памяти. Хранится таблица *dp* размера (n+1) x (m+1), что требует O($n \cdot m$) памяти. Дополнительные переменные: строки, контейнеры для особых символов, строка операций, занимают O(n+m). Итоговая потребность в памяти определяется таблицей и составляет O(n·m).

Тестирование.

Для проверки корректности алгоритма редакционного расстояния с особыми стоимостями было проведено автоматическое тестирование с использованием фреймворка Google Test. Тесты охватывают как типичные ситуации, так и граничные случаи, включая пустые строки, нулевые цены и различные комбинации особых символов. Каждый тест передает функции базовые стоимости, особые цены замены и вставки с конкретными символами и строками *A* и *B*, после чего сверяет полученную последовательность операций с ожидаемой.

Первый тест проверяет случай пустых строк: при *A* = "", *B* = "" алгоритм возвращает пустую последовательность операций, что соответствует нулевой стоимости.

Второй тест проверяет только вставки: *A* = "", *B* = "abc", базовые стоимости 1, особые символы не задействованы. Ожидаемая последовательность — "III". Также проверяется случай с особым символом вставки: если цена вставки для символа 'z' задана высокой, но вставляется другой символ, алгоритм использует базовую цену и выдает корректную последовательность вставок.

Третий тест — только удаления: *A* = "abc", *B* = "", стоимости удалений 5, базовые и особые символы не влияют. Результат — "DDD", то есть три удаления.

Четвертый тест — идентичные строки: A = "hello", B = "hello". Алгоритм выдает "MMMMM", так как все символы совпадают.

Пятый тест — простая замена: A = "a", B = "b". При дешевой замене за 2 и дорогих вставке/удалении за 10 алгоритм выбирает замену "R", а при дорогой замене за 10 и дешевых вставке/удалении за 2 — комбинацию удаления и вставки "DI".

Шестой тест — особая замена. Задан особый символ замены 'a' с низкой ценой 1 при базовой цене замены 10. Ожидается "R". При обратной ситуации, особая цена 'a' завышена до 100, выгоднее становятся удаление и вставка — "DI".

Седьмой тест — особая вставка. Особый символ вставки 'b' с ценой 1, базовые замена 5, удаление 5, вставка 10. При замене 'a' на 'b' стоимость замены равна 5, что дешевле удаления и особой вставки 5+1=6, поэтому результат "R". Если же базовая замена стоит 20, комбинация удаления и особой вставки 5+1=6 дешевле, и алгоритм выдает "DI".

Восьмой тест — равные стоимости всех операций за 1 для строк "ab" и "ba". При равенстве путей алгоритм детерминированно отдает предпочтение замене (проверяется первой), поэтому восстанавливается последовательность "RR".

Девятый тест проверяет особый символ с нулевой ценой замены: замена 'z' на 'w' ничего не стоит. Даже при высоких базовых ценах за 10 выбирается "R", так как замена бесплатна.

Все тесты выполняются автоматически и подтверждают корректность работы алгоритма для заданных условий, включая случаи с особыми стоимостями и граничные ситуации.

Вагнер-Фишер

Задание.

Над строкой ε (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

replace(ε, a, b) – заменить символ a на символ b.

insert(ε, a) – вставить в строку символ a (на любую позицию).

delete(ε, b) – удалить из строки символ b.

Каждая операция может иметь некоторую цену выполнения (положительное число).

Даны две строки A и B, а также три числа, отвечающие за цену каждой операции. Определите последовательность операций (редакционное предписание) с минимальной стоимостью, которые необходимы для превращения строки A в строку B.
Входные данные: первая строка – три числа: цена операции replace, цена операции insert, цена операции delete; вторая строка – A; третья строка – B.

Выходные данные: первая строка – последовательность операций (M – совпадение, ничего делать не надо; R – заменить символ на другой; I – вставить символ на текущую позицию; D – удалить символ из строки); вторая строка – исходная строка A; третья строка – исходная строка B.

Описание алгоритма и оценка его сложности.

Алгоритм Вагнера-Фишера решает задачу поиска минимальной стоимости преобразования строки *A* в строку *B* с использованием трёх операций: замены, вставки и удаления. Каждая операция имеет заданную цену: *replace_cost*, *insert_cost* и *delete_cost* соответственно. Стоимости являются положительными числами, а правило треугольника для них не требуется и не проверяется.

Пусть n — длина строки *A*, m — длина строки *B*. Строится таблица *dp* размера $(n+1)$ x $(m+1)$, где ячейка $dp[i][j]$ содержит минимальную стоимость преобразования первых i символов *A* в первые j символов *B*.

Инициализация границ:

$dp[0][0] = 0$.

$dp[i][0] = i * delete_cost$ — для получения пустой строки из префикса A нужно удалить все i символов.

$dp[0][j] = j * insert_cost$ — для получения префикса B из пустой строки нужно вставить j символов.

Заполнение таблицы происходит построчно для i от 1 до n и j от 1 до m:

Если $A[i-1] == B[j-1]$, то операция не требуется, и $dp[i][j] = dp[i-1][j-1]$.

Иначе вычисляются три варианта:

Замена символа $A[i-1]$ на $B[j-1]$: $dp[i-1][j-1] + replace_cost$.

Вставка символа $B[j-1]$: $dp[i][j-1] + insert_cost$.

Удаление символа $A[i-1]$: $dp[i-1][j] + delete_cost$.

Значение $dp[i][j]$ выбирается как минимум из этих трёх.

После заполнения таблицы минимальная стоимость равна $dp[n][m]$. Для восстановления последовательности операций выполняется обратный ход от $dp[n][m]$ к $dp[0][0]$. На каждом шаге анализируется, какой переход привёл к текущему значению:

При совпадении символов фиксируется операция 'M' без добавления стоимости.

В противном случае сравнивается $dp[i][j]$ с выражениями для замены, вставки и удаления с учётом соответствующих цен. Если значение совпадает с $dp[i-1][j-1] + replace_cost$, записывается 'R'; если с $dp[i][j-1] + insert_cost$ — 'I'; если с $dp[i-1][j] + delete_cost$ — 'D'.

После обхода всей таблицы полученная строка операций переворачивается, давая итоговую последовательность в порядке от начала строки A.

Оценка сложности по времени. Основной цикл двойного перебора имеет сложность $O(n \cdot m)$. Восстановление пути требует $O(n+m)$ операций. Итоговая временная сложность — $O(n \cdot m)$.

Оценка сложности по памяти. Для хранения таблицы dp размера $(n+1)$ x

(m+1) требуется O(n·m) памяти. Дополнительная память под строки и переменные составляет O(n+m). Таким образом, общая потребность в памяти — O(n·m).

Расстояние Левенштейна

Задание.

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк pedestal и stien расстояние Левенштейна равно 7:

Сначала нужно совершить четыре операции удаления символа: pedestal -> stal.

Затем необходимо заменить два последних символа: stal -> stie.

Потом нужно добавить символ в конец строки: stie -> stien.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. (S, $1 \leq |S| \leq 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. (T, $1 \leq |T| \leq 2550$).

Параметры выходных данных:

Одно число L, равное расстоянию Левенштейна между строками S и T.

Описание алгоритма и оценка его сложности.

Расстояние Левенштейна — частный случай редакционного расстояния, в котором стоимости всех трёх операций: замена, вставка, удаление, равны 1. Задача заключается в нахождении минимального количества операций, необходимых для преобразования строки S в строку T.

Пусть n — длина строки S, m — длина строки T. Алгоритм строит таблицу dp размером $(n+1)$ x $(m+1)$, где $dp[i][j]$ — расстояние Левенштейна между первыми i символами S и первыми j символами T.

Граничные условия:

$dp[0][0] = 0$.

$dp[i][0] = i$ для всех i от 0 до n — требуется i удалений, чтобы получить пустую строку из префикса S.

$dp[0][j] = j$ для всех j от 0 до m — требуется j вставок, чтобы получить префикс T из пустой строки.

Заполнение таблицы для $i \geq 1$ и $j \geq 1$:

Если $S[i\text{-}1] == T[j\text{-}1]$, то $dp[i][j] = dp[i\text{-}1][j\text{-}1]$, символы совпадают, дополнительных операций не требуется.

Иначе:

замена: $dp[i\text{-}1][j\text{-}1] + 1$,

вставка: $dp[i][j\text{-}1] + 1$,

удаление: $dp[i\text{-}1][j] + 1$.

Выбирается минимум из трёх значений.

После заполнения всей таблицы искомое расстояние находится в $dp[n][m]$.

Оценка сложности по времени. Алгоритм выполняет двойной цикл по i от 1 до n и j от 1 до m, каждый шаг которого требует O(1) операций. Следовательно, временная сложность составляет $O(n \cdot m)$.

Оценка сложности по памяти. Для хранения таблицы dp размером $(n+1)$ x $(m+1)$ требуется $O(n \cdot m)$ памяти. Дополнительные переменные: строки, индексы, пренебрежимо малы. Таким образом, общая сложность по памяти — $O(n \cdot m)$.